

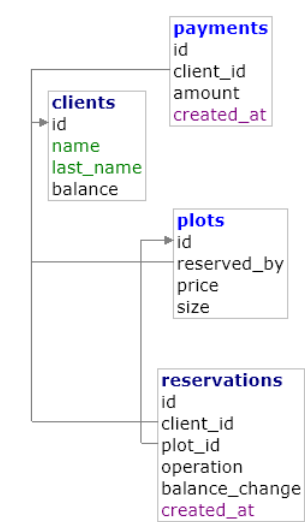
MarsReservation - rezerwacja działek na marsie

Blazej Turczynowicz
Projekt do zarządzania sprzedażą rezerwacji działek na marsie.

- **Technologie backend:** Node.js (Express), PostgreSQL, ORM [sequelize](#).
- **Technologie frontend:** Next.js - React, komponenty Shadcn.

Frontend wykonany w calosci przez AI.

Schemat bazy



Tabele (Modele danych sequelize)

clients - Dane klientow.

```
Client.init(
  {
    id: {
      type: DataTypes.INTEGER,
      allowNull: false,
      primaryKey: true,
      autoIncrement: true,
    },
    name: {
      type: DataTypes.STRING(255),
      allowNull: false,
    },
    last_name: {
      type: DataTypes.STRING(255),
      allowNull: false,
    },
    balance: {
      type: DataTypes.DECIMAL(15, 2),
      allowNull: false,
      defaultValue: sequelize.literal("0.0"),
    },
  },
  {
    sequelize,
    modelName: "Client",
    tableName: "clients",
    timestamps: false,
  },
);
```

plots - Rejestr gruntów.

```
Plot.init(
  {
    id: {
      type: DataTypes.INTEGER,
      allowNull: false,
      primaryKey: true,
      autoIncrement: true,
    },
    reserved_by: {
      type: DataTypes.INTEGER,
      allowNull: true,
      references: {

```

```

        model: "clients",
        key: "id",
      },
    },
    price: {
      type: DataTypes.DECIMAL(15, 2),
      allowNull: false,
      validate: {
        min: 0.01,
      },
    },
    size: {
      type: DataTypes.DECIMAL(15, 2),
      allowNull: false,
      validate: {
        min: 0.01,
      },
    },
  },
  {
    sequelize,
    modelName: "Plot",
    tableName: "plots",
    timestamps: false,
    indexes: [
      {
        fields: ["reserved_by"],
      },
    ],
  },
},
);

```

reservations - Zmian rezerwacji - nowe rezerwacje oraz zmiany statusu.

```

Reservation.init(
{
  id: {
    type: DataTypes.INTEGER,
    allowNull: false,
    primaryKey: true,
    autoIncrement: true,
  },
  client_id: {
    type: DataTypes.INTEGER,
    allowNull: false,
    references: {
      model: "clients",
      key: "id",
    },
  },
  plot_id: {
    type: DataTypes.INTEGER,
    allowNull: false,
    references: {
      model: "plots",
      key: "id",
    },
  },
  operation: {
    type: DataTypes.ENUM("add", "remove"),
    allowNull: false,
  },
  balance_change: {
    type: DataTypes.DECIMAL(15, 2),
    allowNull: false,
  },
  created_at: {
    type: DataTypes.DATE,
    allowNull: true,
    defaultValue: sequelize.literal("CURRENT_TIMESTAMP"),
  },
},
{
  sequelize,
  modelName: "Reservation",
  tableName: "reservations",
  timestamps: false,
  indexes: [
    {
      fields: ["client_id", "plot_id", "created_at"],
    },
  ],
},
);

```

payments - System płatności.

```

Payment.init(
{
  id: {
    type: DataTypes.INTEGER,
    allowNull: false,
    primaryKey: true,
    autoIncrement: true,
  },
  client_id: {
    type: DataTypes.INTEGER,
    allowNull: false,
    references: {
      model: "clients",
      key: "id",
    },
  },
  amount: {
    type: DataTypes.DECIMAL(15, 2),
    allowNull: false,
  },
  created_at: {
    type: DataTypes.DATE,
    allowNull: true,
    defaultValue: sequelize.literal("CURRENT_TIMESTAMP"),
  },
},
{
  sequelize,
  modelName: "Payment",
  tableName: "payments",
  timestamps: false,
  indexes: [
    {
      fields: ["client_id", "created_at"],
    },
  ],
},
);

```

Procedury

p_reserve - Rezerwuje działke.

```

CREATE PROCEDURE p_reserve(p_client_id INT, p_plot_id INT)
LANGUAGE plpgsql
AS $$
DECLARE v_client_balance DECIMAL(15, 2);
DECLARE v_plot_price DECIMAL(15, 2);
DECLARE v_balance_change DECIMAL(15, 2);
DECLARE v_is_available BOOLEAN;
BEGIN
  SELECT price, (reserved_by IS NULL)
  INTO v_plot_price, v_is_available
  FROM plots
  WHERE id = p_plot_id
  FOR UPDATE;

  SELECT balance INTO v_client_balance FROM clients WHERE id = p_client_id FOR UPDATE;

  IF NOT v_is_available THEN
    RAISE EXCEPTION 'Plot % is already reserved.', p_plot_id;
  END IF;

  IF v_client_balance >= v_plot_price THEN
    INSERT INTO reservations (client_id, plot_id, operation, balance_change)
    VALUES (p_client_id, p_plot_id, 'add', -v_plot_price);

    UPDATE plots SET reserved_by = p_client_id WHERE id = p_plot_id;
  ELSE
    RAISE EXCEPTION 'Not enough balance to reserve the plot.';
  END IF;

  COMMIT;

  EXCEPTION WHEN OTHERS THEN
    ROLLBACK;
    RAISE;
END;
$$;

```

p_remove_reservation - Usuwa rezerwację działki.

```
CREATE PROCEDURE p_remove_reservation(p_client_id INT, p_plot_id INT)
LANGUAGE plpgsql
AS $$
DECLARE v_is_owned BOOLEAN;
DECLARE v_previous_balance_change DECIMAL(15, 2);
BEGIN
    SELECT (reserved_by = p_client_id) INTO v_is_owned
    FROM plots
    WHERE id = p_plot_id
    FOR UPDATE;

    IF NOT v_is_owned THEN
        RAISE EXCEPTION 'Plot % is not reserved by client %.', p_plot_id, p_client_id;
    END IF;

    SELECT balance_change INTO v_previous_balance_change
    FROM reservations
    WHERE client_id = p_client_id AND plot_id = p_plot_id AND operation = 'add'
    FOR UPDATE;

    INSERT INTO reservations (client_id, plot_id, operation, balance_change)
    VALUES (p_client_id, p_plot_id, 'remove', -v_previous_balance_change);

    UPDATE plots SET reserved_by = NULL WHERE id = p_plot_id;

    COMMIT;
EXCEPTION WHEN OTHERS THEN
    ROLLBACK;
    RAISE;
END;
$$;
```

Widoki

v_available_plots - Widok tylko wolnych gruntów

```
CREATE OR REPLACE VIEW v_available_plots AS
SELECT * FROM plots
WHERE reserved_by IS NULL;
```

v_client_summary - Podsumowanie klienta

```
CREATE OR REPLACE VIEW v_client_summary AS
WITH
client_plots AS (
    SELECT
        reserved_by AS client_id,
        COUNT(id) AS plots_owned_count,
        COALESCE(SUM(size), 0) AS total_size
    FROM plots
    GROUP BY reserved_by
),
client_reservations AS (
    SELECT
        client_id,
        SUM(balance_change) AS total_paid
    FROM reservations
    GROUP BY client_id
)
SELECT
    c.id AS client_id,
    c.name || ' ' || c.last_name AS full_name,
    c.balance AS current_account_balance,
    COALESCE(cp.plots_owned_count, 0) AS plots_owned_count,
    COALESCE(cp.total_size, 0) AS total_size,
    COALESCE(cr.total_paid, 0) AS total_paid
FROM clients c
LEFT JOIN client_plots cp ON c.id = cp.client_id
LEFT JOIN client_reservations cr ON c.id = cr.client_id;
```

Triggery

trg_plot_reserved - Zmienia stan rezerwacji działki po pojawieniu się zmiany w **reservations**.

```
CREATE OR REPLACE FUNCTION f_update_reserved() RETURNS TRIGGER AS $$
BEGIN
    UPDATE plots
    SET reserved_by = NEW.client_id
```

```

        WHERE id = NEW.plot_id;
        RETURN NEW;
    END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_plot_reserved
AFTER INSERT ON reservations
FOR EACH ROW
EXECUTE FUNCTION f_update_reserved();

```

trg_client_balance - Aktualizuje posiadane srodki na koncie klienta po zmianie w systemie platnosci payments.

```

CREATE OR REPLACE FUNCTION f_update_user_balance() RETURNS TRIGGER AS $$
BEGIN
    UPDATE clients
    SET balance = balance + NEW.amount
    WHERE id = NEW.client_id;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_client_balance
AFTER INSERT ON payments
FOR EACH ROW
EXECUTE FUNCTION f_update_user_balance();

```

trg_update_payments - Aktualizuje posiadane srodki na koncie klienta po zmianie w systemie platnosci payments.

```

CREATE OR REPLACE FUNCTION f_update_payments() RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO payments (client_id, amount)
    SELECT NEW.client_id, NEW.balance_change;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_update_payments
AFTER INSERT ON reservations
FOR EACH ROW
EXECUTE FUNCTION f_update_payments();

```

Funkcje

fn_plots_sold_between - Raport sprzedazy rezerwacji w danym przedziale dat.

```

CREATE OR REPLACE FUNCTION fn_plots_sold_between(p_start_date TIMESTAMP, p_end_date TIMESTAMP)
RETURNS TABLE (
    time_of_transaction TIMESTAMP,
    client_name VARCHAR(255),
    plot_id INT,
    amount DECIMAL(15, 2),
    size DECIMAL(15, 2)
) AS $$
SELECT
    r.created_at,
    (c.name || ' ' || c.last_name)::VARCHAR(255),
    r.plot_id,
    ABS(r.balance_change)::DECIMAL(15, 2),
    p.size
FROM reservations r
JOIN clients c ON r.client_id = c.id
JOIN plots p ON r.plot_id = p.id
WHERE r.operation = 'add'
    AND r.created_at BETWEEN p_start_date AND p_end_date
    AND NOT EXISTS (
        SELECT 1 FROM reservations r2
        WHERE r2.client_id = r.client_id
        AND r2.plot_id = r.plot_id
        AND r2.operation = 'remove'
        AND r2.created_at > r.created_at
    )
ORDER BY r.created_at DESC;
$$ LANGUAGE sql;

```

Uruchomienie

W plikach migracji jest uwzględnione seedowanie bazy, tworzy kilku klientów i kilka działek

- Start kontenera docker - `docker-compose up -d`
- (Pierwszy start) - załadować migracje bazy `docker compose exec backend npm run db:migrate -- --config database/config/config.js`

Komunikacja z serwerem - endpointy

Metoda	Endpoint	Opis	Body / Query
GET	/clients	Pobiera listę wszystkich klientów.	brak
GET	/clients/:clientId	Pobiera dane konkretnego klienta.	:clientId (URL)
PATCH	/clients/:clientId	Aktualizuje dane klienta.	{ name, lastName, balance }
POST	/add-client	Tworzy nowego klienta.	{ name, lastName }
GET	/client-summary/:clientId	Pobiera podsumowanie dla klienta (widok SQL).	:clientId (URL)
GET	/plots	Pobiera listę dostępnych parceli (widok SQL).	brak
GET	/plots/all	Pobiera listę wszystkich parceli.	brak
GET	/plots/:plotId	Pobiera dane konkretnej parceli.	:plotId (URL)
POST	/plots	Dodaje nową parcelę.	{ price, size, reservedBy }
PATCH	/plots/:plotId	Aktualizuje dane parceli.	{ price, size, reservedBy }
POST	/reserve	Wywołuje procedurę rezerwacji działki.	{ clientId, plotId }
POST	/remove-reservation	Wywołuje procedurę usunięcia rezerwacji.	{ clientId, plotId }
GET	/reservations	Historia rezerwacji (z filtrowaniem).	Query: clientId, plotId
GET	/payments	Historia płatności (z filtrowaniem).	Query: clientId
GET	/reports/plots-sold	Raport sprzedanych parceli (funkcja SQL).	Query: startDate, endDate